GitHub & Version Control

RamRodz Robotics · Team 6366 · Programming Worksheet — Lesson 8

RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS · SIMPSONVILLE, SC

Focus Check-In — Struggling | Okay | Ready

TODAY'S BIG QUESTION

If two programmers both change the robot code at the same time, how do you stop them from overwriting each other's work — and how do you go back to a working version when something breaks?

Parking Lot — Random Thoughts

1 Key Words — Check Off When Explained

Git
A tool that tracks every change you make to code — like an undo history that never expires, even across a whole team.

GitHub
A website that stores your Git repository online so the whole team can access it from any computer.

Repository (Repo)
The folder that holds your project AND its complete history of every change ever saved.

Commit
A saved snapshot of your changes with a message describing what you did. Think: save + label.

Push
Upload your local commits to GitHub so teammates can see them.

Pull
Download your teammates' commits from GitHub into your local copy.

Clone
Make a full local copy of a GitHub repository on your computer for the first time.

Branch
A separate copy of the code where you can experiment safely — without touching the working main version.

Main
The default branch — the "official" working version of your robot code. You protect it by working on branches first.

Merge
Combine the changes from one branch into another — usually bringing a feature branch back into main.

2 Why Does a Robot Team Need Version Control?

Without Git, robot teams share code by emailing files, copying USB drives, or just hoping nobody edits the same thing at the same time. At a competition, that breaks robots. Here's why Git exists:

Without Git	With Git & GitHub
Overwrite a teammate's fix without knowing	Git detects conflicts and asks you to resolve them
Break auto routine 20 minutes before a match — no way back	Roll back to any previous commit instantly
Two programmers can't work at the same time	Each works on their own branch; merge when done
"Which version did we deploy?" — nobody knows	Every commit is labeled, dated, and traceable

- [ ] I can name two reasons why a robot team needs version control.

🖍️ **Quick Question — What's the worst thing that could happen to robot code the day before competition if the team doesn't use Git?**

3 Creating a GitHub Repository

A repository starts life on GitHub as an empty container. Once created, every team member clones it to their own computer. Here's what that creation screen looks like:

⬡ github.com / New repository

Create a new repository

Repository name

RamRodz2025Robot

Visibility

◉ **Private** — only team members can see it

○ Public — anyone on the internet can view it

☑ *Add a README file* ☑ *Add .gitignore → WPILib Java template*

Create repository

WPILib .gitignore — When you create the repo, choose the *WPILib Java* .gitignore template. This tells Git to ignore build output folders (like `build/`) and IDE settings that don't belong in shared code.

☐ I watched the repo get created and I know what the Repository Name, Visibility, and .gitignore fields do.

✏️ Fill In — What two things should you check when creating an FRC robot repository?

_____

_____

4 Clone the Repo & Open in VS Code

Cloning downloads the full repository — all files AND all history — to your computer for the first time. You only do this once per machine.

STEP 1 — COPY THE URL FROM GITHUB

On the repo's main page click the green **Code** button → copy the HTTPS URL. It looks like:
`https://github.com/RamRodz6366/RamRodz2025Robot.git`

STEP 2 — CLONE VIA VS CODE (EASIEST WAY)

Press `Ctrl` + `Shift` + `P` to open the Command Palette → type `Git: Clone` → paste the URL → choose where to save it on your computer.

STEP 3 — OPEN THE REPO IN VS CODE

After the clone finishes, VS Code shows a pop-up: *"Would you like to open the cloned repository?"* — click **Open**. The repo is now your workspace and Git is already connected.

Plain English: Cloning is like downloading a ZIP of all the code, but smarter — it also brings the entire commit history and keeps the link to GitHub so every future push and pull works automatically.

VS Code Source Control Panel — After opening the cloned folder in VS Code, click the branch icon in the left sidebar (or press `Ctrl` + `Shift` + `G`) to open the Source Control panel. This is where you stage, commit, push and pull without typing terminal commands.

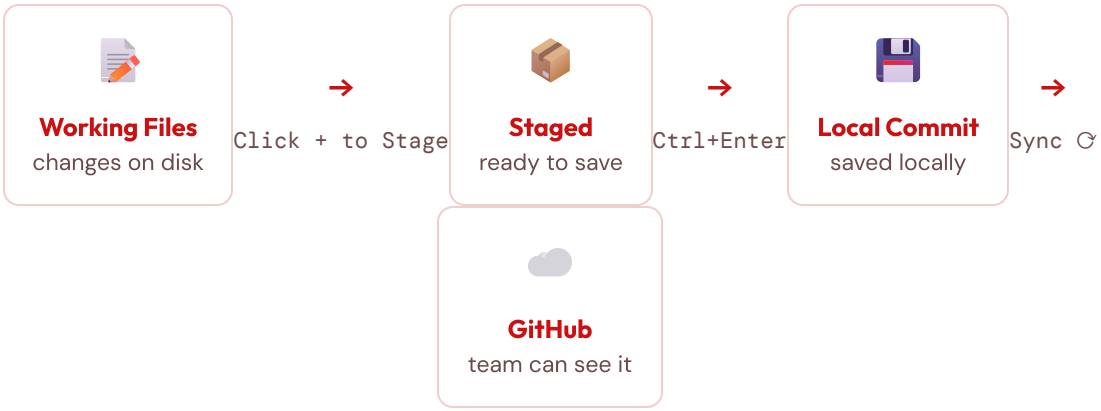
- [ ] I can clone a repo in VS Code and open the Source Control panel.

Fill In — What VS Code keyboard shortcut opens the Source Control panel?

_____

5 The Daily Workflow: Stage → Commit → Push

Every time you finish a chunk of work, you run through these four steps. You'll do this dozens of times every build season.



STEP A — OPEN SOURCE CONTROL TO SEE CHANGES

Press `Ctrl` + `Shift` + `G` or click the branch icon in the left sidebar. Every modified file appears listed under *Changes* — you can click any file to see exactly what you edited.

STEP B — STAGE YOUR FILES

Hover over a file and click the **+** icon to stage it, or click the **+** next to the *Changes* heading to stage everything at once. Staged files move up to the *Staged Changes* section.

STEP C — WRITE A COMMIT MESSAGE AND COMMIT

Click the text box at the top of the Source Control panel that says *"Message (Ctrl+Enter to commit)"*. Type a short description — e.g. `Add scoreBall() sequence command` — then press `Ctrl` + `Enter`.

STEP D — PUSH TO GITHUB

Click the **Sync Changes** ⟳ button in the Source Control panel — or click the branch indicator in the bottom-left status bar which shows the number of pending commits. One click uploads your commit to GitHub.

Plain English: Think of **stage** as putting files in a box, **commit** as sealing the box with a label, and **Sync** ⟳ as shipping it to the team warehouse on GitHub. Before you start each session, **Sync first** to pull any boxes your teammates shipped.

START OF EVERY SESSION — PULL FIRST!

Click **Sync** ↻ in the Source Control panel before you write a single line of code. This pulls your teammates' latest commits. VS Code shows a notification if anything was pulled. Make it a reflex: *open VS Code → click Sync → then start coding*.

Good commit messages: Write what you changed and why, not just "fixed stuff". Good: *"Add scoreBall() sequence command"*. Bad: *"stuff"*. Your teammates and future-you will thank you when searching history at 10pm before regionals.

- [ ] I can explain all four steps — pull, stage, commit, push — and say what order to do them in.

Sequence It — Number these VS Code steps in the correct order (1–4):

_____

_____

_____

_____

6 Branches & Merging

A branch is a parallel copy of your code. You work on the branch without touching **main**. When it works, you merge it back. If it breaks, you delete the branch and main is still perfect.

Branch Lifecycle



Branch created from C2 → two feature commits → merged back to main as M

STEP A — CREATE A NEW BRANCH IN VS CODE

Click the **branch name** in the bottom-left status bar (it shows `main`). A menu appears at the top — choose *"Create New Branch..."*, type a name like `auto-2025`, and press `Enter`. VS Code automatically switches to your new branch — the status bar updates to show its name.

STEP B — WORK ON THE BRANCH AND COMMIT

Make your code changes, then use the same Source Control steps you already know: stage with **+**, write a message, press `Ctrl` + `Enter` to commit. Click **Sync** ⟳ to push the branch to GitHub so your teammate can see it too.

STEP C — SWITCH BRANCHES

Click the branch name in the status bar again. All your branches are listed — click one to switch to it instantly. VS Code updates all your open files to match that branch. You can hop between `main` and `auto-2025` as many times as you want.

STEP D — MERGE BACK TO MAIN WHEN READY

First switch to `main` via the status bar. Then press `Ctrl` + `Shift` + `P` → type *"Git: Merge Branch..."* → choose `auto-2025`. VS Code brings all your branch commits into main. Hit **Sync** ⟳ to push the merged main to GitHub.

Plain English: **Branches are free experiments.** Create one, break things, try things. Main stays safe the whole time. When your branch works — merge it in. If it doesn't — delete the branch and walk away. Main never knew.

Branch name in the status bar: The bottom-left corner of VS Code always shows which branch you're on. It's your command center — click it to create a branch, switch branches, or see all branches at a glance. You never need to leave VS Code.

- [ ] I can explain why you'd make a branch instead of editing main directly, and what `merge` does.

🖍️ Explain It — In your own words, why is working on a branch safer than editing main directly?

📋 VS Code Git Quick Reference

`Ctrl+Shift+G`	Open Source Control panel — see all changed files
`Sync ⟳ (pull)`	Download teammates' latest commits — do this first every session
`+ next to a file`	Stage that file (moves it to Staged Changes)
`+ next to Changes`	Stage ALL changed files at once
`Message box → Ctrl+Enter`	Type your commit message and save the snapshot
`Sync ⟳ (push)`	Upload your commits to GitHub
`Ctrl+Shift+P → Git: Clone`	Clone a GitHub repo to your computer for the first time
`Status bar branch name`	Click to switch branches or create a new one
`Ctrl+Shift+P → Git: Merge Branch`	Merge a branch into your currently active branch

🎯 Robot Scenario — Competition Day Branch Strategy

🏆 It's the Night Before Regionals

Your robot's teleop code is solid and merged into **main**. Your lead programmer just had an idea for a new 3-ball autonomous routine. You want to try it — but if it crashes the robot, you need to be able to get back to the working code instantly. The drive team is watching and very nervous.

- You have the teleop code committed and pushed to main
- You want to add and test a new autonomous routine
- If it doesn't work by 10pm, you ship the working teleop code
- Two programmers want to work on it at the same time

🖍️ Step 1 — How do you create the new branch in VS Code, and what does it create?

🖍️ Step 2 — You and a teammate both push commits to `auto-2025`. What do you each do in VS Code at the start of your session to get the other's work?

🖍️ Step 3 — It works! Describe the VS Code steps to bring the auto branch into main and push to GitHub.

🖍️ What If — It's 10pm and the auto routine still doesn't work. How do you make sure the drive team gets the working teleop code deployed? (No commands needed — just explain the strategy.)

- [ ] I can describe the branch-and-merge competition-day strategy and explain why main always stays safe.

✅ End of Lesson — I Can...

- [ ] Create a new GitHub repository with a README and WPILib .gitignore.
- [ ] Clone a repository into VS Code and open the Source Control panel.
- [ ] Run the daily workflow: pull → stage → commit with a message → push.
- [ ] Create a branch, commit changes to it, then merge it back into main.
- [ ] Explain why branches protect the working robot code during development.